YENEPOYA
(Deemed to be University)

YENEPOYA UNIVERSITY

LOW LEVEL DESIGN (LLD) DOCUMENT
ON
Edge AI for Healthcare Realtime Monitoring

Student Name: Rhidhik KS

Registration No: 23BCAICD092

Campus ID: 25287

Industry Mentor: Sumit Kumar Shukla

Guided By: Ms. Deeksha Shetty

LOW LEVEL DESIGN

ICU Monitoring System

using Edge AI & Firebase Cloud

Document Type	Low Level Design (LLD)
Version	1.0
Date	April 2026
Domain	Healthcare / IoT / Edge AI

1. Module: Arduino Firmware (patient_monitoring.ino)

1.1 Purpose

The Arduino firmware is the lowest layer of the system. It initializes and reads the DHT sensor, serializes the data as a comma-separated string, transmits it over the hardware UART, and controls an LED as a hardware-level alert.

1.2 Pin Configuration

DHTPIN	Digital Pin 2 — Data line for DHT11/DHT22 sensor (with pull-up resistor)
LED_PIN	Digital Pin 13 — Built-in LED used for visual alert
Serial (TX)	Digital Pin 1 (UART TX) — USB Serial at 9600 baud

1.3 Setup Function

The setup() function runs once on power-on or reset. It initializes all peripherals before the main loop begins.

```
void setup() {
  Serial.begin(9600);    // Open UART at 9600 baud
  dht.begin();           // Initialize DHT sensor
  pinMode(LED_PIN, OUTPUT);  // Set LED pin as output
}
```

1.4 Main Loop Logic

The loop() function runs continuously. It reads sensor values, validates them, serializes the data, triggers the LED alert, then delays 2 seconds.

```
void loop() {
  float temp = dht.readTemperature();   // Celsius
  float hum  = dht.readHumidity();      // Percent

  if (isnan(temp) || isnan(hum)) return; // Guard: skip bad readings

  Serial.print(temp);   // Write float
  Serial.print(',');    // Delimiter
  Serial.println(hum);  // Write float + newline

  if (temp > 30 || hum > 60) {
    digitalWrite(LED_PIN, HIGH);  // Alert ON
  } else {
    digitalWrite(LED_PIN, LOW);   // Alert OFF
  }
  delay(2000);  // 2-second sampling interval
```

```
}
```

1.5 Alert Thresholds (Hardware Level)

Parameter	Threshold	Action
Temperature	> 30°C	LED ON (Pin 13 HIGH)
Humidity	> 60%	LED ON (Pin 13 HIGH)
Both Normal	All below threshold	LED OFF (Pin 13 LOW)

NOTE: Hardware thresholds (30°C / 60%) are more sensitive than software thresholds (38°C / 80%) to provide an early physical alert before critical conditions escalate.

2. Module: Edge Monitor (edge_monitor.py)

2.1 Purpose

The edge monitor is the central processing unit of the system. It reads sensor data from Arduino via serial, performs AI inference using the TFLite model, classifies patient status, and uploads structured data to Firebase Realtime Database.

2.2 Initialization Sequence

Three objects are initialized at startup before the main loop begins:

```
# 1. Open serial port (blocks until Arduino is ready)
ser = serial.Serial('COM3', 9600, timeout=1)

# 2. Initialize Firebase (guard prevents double initialization)
if not firebase_admin._apps:
    cred = credentials.Certificate('firebase_key.json')
    firebase_admin.initialize_app(cred, {
        'databaseURL': 'https://icu-monitoring-system-bc1c3...'
    })

# 3. Load TFLite model and allocate tensor buffers
interpreter = tf.lite.Interpreter(model_path='model.tflite')
interpreter.allocate_tensors()
input_details  = interpreter.get_input_details()
output_details = interpreter.get_output_details()
```

2.3 Main Loop — Step-by-Step

Step 1: Read Serial Line

```
line = ser.readline().decode().strip()
if not line: continue  # Skip empty frames
```

readline() blocks until a newline character is received or timeout=1 second elapses. The raw bytes are decoded to UTF-8 and stripped of whitespace.

Step 2: Parse CSV

```
values = line.split(',')
if len(values) < 2: continue  # Guard: require exactly 2 fields
temp = float(values[0])
hum  = float(values[1])
```

Step 3: TFLite Inference

```
# Normalize: temp range [0..50], hum range [0..100]
```

```
input_data = np.array([[temp/50.0, hum/100.0]], dtype=np.float32)

interpreter.set_tensor(input_details[0]['index'], input_data)
interpreter.invoke()  # Run forward pass
ai_score = float(interpreter.get_tensor(output_details[0]['index'])[0][0])
```

The model expects a float32 tensor of shape [1, 2]. Normalization divides temperature by 50 and humidity by 100 to map values into [0.0, 1.0] range consistent with training.

Step 4: Status Classification

```
if   temp > 38 or hum > 80:  status = 'CRITICAL'
elif temp > 30 or hum > 70:  status = 'WARNING'
else:                         status = 'SAFE'
```

Rule-based thresholds override the AI score for final status determination. The AI score is stored separately as supplementary information for trend analysis.

Step 5: Patient Assignment

```
patients = ['P001', 'P002', 'P003']
patient_id = patients[index]
index = (index + 1) % 3  # Round-robin: 0 -> 1 -> 2 -> 0
```

Step 6: Firebase Write

```
ref = db.reference(f'patients/{patient_id}')
ref.set({
    'temp':     temp,
    'humidity': hum,
    'ai_score': ai_score,
    'status':   status,
    'time':     str(datetime.now())
})
```

2.4 Error Handling

All loop operations are wrapped in a single try/except block. Any exception (serial read error, Firebase timeout, value conversion error) is caught, printed to stdout, and the loop continues without crashing.

```
while True:
    try:
        # ... all processing ...
    except Exception as e:
        print('Error:', e)  # Log and continue
```

NOTE: The current implementation catches all exceptions broadly. In production, distinct handlers for SerialException, FirebaseError, and ValueError would provide better diagnostics.

2.5 Configuration Parameters

Parameter	Type	Default Value	Description
COM3	str	COM3	Serial port identifier; change to /dev/ttyUSB0 on Linux
9600	int	9600	Baud rate; must match Arduino Serial.begin()
timeout	float	1.0 sec	Serial read timeout; prevents blocking indefinitely
patients	list	[P001,P002,P003]	Patient ID pool for round-robin assignment
time.sleep(2)	float	2 seconds	Inter-reading delay between Firebase writes

3. Module: AI Model (train_model.py + convert.py)

3.1 Training Data Generation

The training dataset is synthetically generated using NumPy random sampling. Real ICU data is not used in this version.

```
temps = np.random.uniform(25, 40, 2000)  # 2000 samples, 25-40°C
hums  = np.random.uniform(30, 90, 2000)  # 2000 samples, 30-90%

# Binary label: 1 = WARNING, 0 = SAFE
y = np.where((temps > 35) | (hums > 65), 1, 0)
```

3.2 Feature Normalization

```
temp_norm = (temps - 25) / (40 - 25)  # Map [25..40] -> [0..1]
hum_norm  = (hums  - 30) / (90 - 30)  # Map [30..90] -> [0..1]
X = np.column_stack((temp_norm, hum_norm))  # Shape: [2000, 2]
```

Note: Training normalization uses (value - min) / range, while inference normalization uses simpler division (temp/50, hum/100). These are functionally different but both scale the inputs to similar ranges.

3.3 Model Architecture

Layer	Configuration	Output Shape
Input	2 features (temp_norm, hum_norm)	[batch, 2]
Dense 1	16 units, ReLU activation	[batch, 16]
Dense 2	8 units, ReLU activation	[batch, 8]
Dense 3 (Output)	1 unit, Sigmoid activation	[batch, 1]

3.4 Training Configuration

Optimizer	Adam (adaptive learning rate)
Loss Function	Binary Cross-Entropy (binary classification)
Metric	Accuracy
Epochs	50
Dataset Size	2000 samples
Output File	model.h5 (Keras HDF5 format)

3.5 TFLite Conversion (convert.py)

```
model = tf.keras.models.load_model('model.h5')
converter = tf.lite.TFLiteConverter.from_keras_model(model)

# Apply default quantization (INT8 weights, reduces model size)
converter.optimizations = [tf.lite.Optimize.DEFAULT]

tflite_model = converter.convert()
with open('model.tflite', 'wb') as f:
    f.write(tflite_model)  # Output: ~2.8KB
```

The DEFAULT optimization applies post-training quantization which reduces the model from float32 (~30KB in .h5) to INT8 weights (~2.8KB in .tflite) with minimal accuracy loss.

3.6 Inference Input/Output Specification

Input Tensor Shape	[1, 2] — float32
Input [0]	temperature / 50.0 (e.g., 28.5°C → 0.57)
Input [1]	humidity / 100.0 (e.g., 65% → 0.65)
Output Tensor Shape	[1, 1] — float32
Output Range	[0.0, 1.0] — higher = higher risk
Interpretation	Sigmoid probability score; not directly used for status (rules override)

4. Module: Dashboard (dashboard.py)

4.1 Purpose

The Streamlit dashboard provides the real-time user interface for ICU staff. It fetches data from Firebase, renders patient status cards, displays live trend graphs for temperature and humidity, and allows per-patient drill-down.

4.2 Initialization and Configuration

```
st.set_page_config(layout='wide')   # Use full browser width

# Auto-refresh: triggers full Streamlit re-run every 1000ms
st_autorefresh(interval=1000, key='refresh')

# Firebase initialization (guarded against duplicate init)
if not firebase_admin._apps:
    cred = credentials.Certificate('firebase_key.json')
    firebase_admin.initialize_app(cred, {'databaseURL': '...'})
```

4.3 Data Fetch and Transformation

```
ref  = db.reference('patients')    # Root patients node
data = ref.get()                   # Returns dict of all patients

# Stop rendering if no data yet
if not data:
    st.warning('Waiting for Firebase data...')
    st.stop()

# Flatten Firebase dict into DataFrame rows
rows = []
for pid, values in data.items():
    rows.append({
        'patient_id': pid,
        'temp':       values.get('temp'),
        'hum':        values.get('humidity'),
        'status':     values.get('status'),
        'prediction': values.get('ai_score')
    })
df = pd.DataFrame(rows)
```

4.4 Patient Status Cards

Each patient is rendered in a separate Streamlit column with an HTML div styled according to their current status. CRITICAL cards have a CSS blink animation.

```
cols = st.columns(len(df))
```

```
for i, row in df.iterrows():
    cls = 'critical' if row['status'] == 'CRITICAL' else \
          'warning'  if row['status'] == 'WARNING'  else 'safe'

    cols[i].markdown(f'''
    <div class="card {cls}">
        <h3>{row['patient_id']}</h3>
        Temp: {row['temp']:.1f} °C<br>
        Humidity: {row['hum']:.1f} %<br>
        AI Score: {row['prediction']:.2f}<br>
        <b>{row['status']}</b>
    </div>''', unsafe_allow_html=True)
```

4.5 CSS Status Classes

CSS Class	Background Color	Visual Effect
.safe	#1f6f4a (dark green)	Static display
.warning	#8a6d1a (dark amber)	Static display
.critical	#7a1c1c (dark red)	CSS blink animation at 1s interval

4.6 Trend Graph Generation

The current version generates simulated history by adding small random noise to the latest readings across 20 time points. This creates a plausible-looking trend without requiring historical data storage.

```
plot_data = []
for i in range(20):  # 20 history points
    for _, row in df.iterrows():
        plot_data.append({
            'time':       pd.Timestamp.now() - pd.Timedelta(seconds=(20-i)),
            'patient_id': row['patient_id'],
            'temp':       row['temp'] + np.random.uniform(-0.2, 0.2),
            'hum':        row['hum']  + np.random.uniform(-0.5, 0.5)
        })

df_plot = pd.DataFrame(plot_data)

# Temperature multi-line chart
fig1 = px.line(df_plot, x='time', y='temp',
               color='patient_id', template='plotly_dark')
st.plotly_chart(fig1, use_container_width=True)
```

NOTE: The random noise approach is for demonstration only. In production, the edge monitor should push timestamped readings to a history path in Firebase (e.g. patients/P001/history/{timestamp}) for real trend data.

4.7 Patient Selector

```
selected = st.selectbox('Select Patient', df['patient_id'])
st.subheader(f'{selected} Details')
st.write(df[df['patient_id'] == selected])  # Display filtered DataFrame
```

5. Firebase Realtime Database Schema

5.1 Database Tree Structure

```
{
  'patients': {
    'P001': {
      'temp':     28.5,
      'humidity': 65.2,
      'ai_score': 0.73,
      'status':   'WARNING',
      'time':     '2026-04-22 10:30:45.123456'
    },
    'P002': {
      'temp':     22.1,
      'humidity': 55.0,
      'ai_score': 0.12,
      'status':   'SAFE',
      'time':     '2026-04-22 10:30:47.456789'
    },
    'P003': {
      'temp':     39.2,
      'humidity': 82.0,
      'ai_score': 0.97,
      'status':   'CRITICAL',
      'time':     '2026-04-22 10:30:49.789012'
    }
  }
}
```

5.2 Field Validation Rules

Field	Type	Valid Range	Notes
temp	Float	0.0 – 60.0	°C; DHT11 range: 0–50°C
humidity	Float	0.0 – 100.0	% relative humidity
ai_score	Float	0.0 – 1.0	Sigmoid output from TFLite model
status	String	SAFE|WARNING|CRITICAL	Enum; derived by rule engine
time	String	ISO 8601 datetime	Python datetime.now() as string

5.3 Firebase Security Rules (Recommended)

```
{
  'rules': {
    'patients': {
      '$patient_id': {
        '.write': 'auth != null',  // Only authenticated writes
        '.read':  'auth != null'   // Only authenticated reads
      }
```

```
        }
    }
}
```

6. Error Handling & Edge Cases

Scenario	Detection	Current Handling	Recommended Fix
DHT read fails (NaN)	isnan() check in Arduino	return; skip iteration	Log to Serial for debugging
Empty serial line	if not line: continue	Skip empty frames	Count skipped frames
CSV parse fails	len(values) < 2 check	continue (skip)	Log malformed line content
Firebase timeout	try/except Exception	Print error, continue	Implement exponential backoff
No Firebase data	if not data: st.stop()	Show warning, halt render	Show stale data with timestamp
Serial port not found	try/except Exception	Crash at startup	Prompt user for correct port
Model file missing	TFLite load exception	Crash at startup	Add file existence check

7. Deployment & Setup Guide

7.1 Prerequisites

- Python 3.8+ installed
- Node.js / Arduino IDE for firmware upload
- Firebase project created with Realtime Database enabled
- Service Account JSON key downloaded as firebase_key.json

7.2 Installation

```
# Install all Python dependencies
pip install -r requirements.txt

# Contents of requirements.txt:
# streamlit
# pandas
# plotly
# numpy
# tensorflow
# pyserial
# firebase-admin
# streamlit-autorefresh
```

7.3 Hardware Setup

- Connect DHT11/DHT22 Data pin to Arduino Digital Pin 2 (with 10kΩ pull-up to VCC)
- Connect DHT11/DHT22 VCC to 5V, GND to GND
- LED on Digital Pin 13 (Arduino built-in LED is sufficient)
- Upload patient_monitoring.ino via Arduino IDE
- Note the COM port assigned to the Arduino (Device Manager on Windows, /dev/ttyUSB0 on Linux)

7.4 Running the System

```
# Terminal 1: Start edge monitor
# (Update COM3 in edge_monitor.py if needed)
python edge_monitor.py

# Terminal 2: Start dashboard
streamlit run dashboard.py
# Dashboard opens at: http://localhost:8501
```

7.5 Retraining the AI Model

```
# Step 1: Train the model
python train_model.py   # Produces model.h5

# Step 2: Convert to TFLite
python convert.py       # Produces model.tflite
```

8. Known Limitations & Future Improvements

8.1 Current Limitations

- Single sensor serves all 3 patients in round-robin — not true multi-patient hardware
- Dashboard uses fake random noise for trend graphs — no real historical data
- Firebase only stores the latest reading (no history node)
- AI model trained on synthetic data — not on real ICU environmental data
- Serial port is hardcoded as COM3 — not portable across machines
- No authentication or access control on the dashboard

8.2 Recommended Enhancements

Historical Storage	Write readings to patients/{id}/history/{timestamp} in Firebase for real time-series data
Multi-sensor Hardware	Deploy one Arduino+DHT per patient bed; edge monitor subscribes to multiple serial streams
Advanced Sensors	Add MAX30102 (SpO2 + Heart Rate) and BMP280 (barometric pressure) for richer features
Model Improvement	Train on real clinical data; use multi-class output (SAFE/WARNING/CRITICAL) instead of binary
Mobile App	React Native app consuming Firebase RTDB for push notifications and on-call alerts
Cloud Deployment	Containerize edge monitor with Docker; deploy dashboard to AWS/GCP for remote access
Security	Implement Firebase Auth, HTTPS, and encrypted serial communication
Configuration File	Externalize COM port, baud rate, patient IDs, thresholds to a config.yaml or .env file